

Propiedades de la Sumatoria Clave para la Integral Definida

Resumen Conceptual

September 1, 2026

Abstract

Este documento resume las propiedades esenciales de la sumatoria que permiten transformar sumas finitas en límites, facilitando la comprensión de la integral definida como el límite de una suma de Riemann.

1 Introducción

La integral definida de una función $f(x)$ en el intervalo $[a, b]$ se define como:

$$\int_a^b f(x) dx = \lim_{n \rightarrow \infty} \sum_{i=1}^n f(x_i^*) \Delta x$$

donde:

- $\Delta x = \frac{b-a}{n}$ es el ancho de cada subintervalo.
- x_i^* es un punto muestra en el i -ésimo subintervalo.

Para evaluar este límite, necesitamos dominar las propiedades algebraicas de la sumatoria.

2 Propiedades Fundamentales de la Sumatoria

A continuación se presentan las propiedades más útiles, con ejemplos que conectan directamente con la integral definida.

2.1 Linealidad

$$\begin{aligned} \sum_{i=1}^n (a_i + b_i) &= \sum_{i=1}^n a_i + \sum_{i=1}^n b_i \\ \sum_{i=1}^n c \cdot a_i &= c \sum_{i=1}^n a_i \quad (c \text{ constante}) \end{aligned}$$

Esta propiedad permite separar sumas y extraer constantes, lo que es esencial al expandir $f(x_i^*)$.

2.2 Suma de una constante

$$\sum_{i=1}^n c = c \cdot n$$

Útil cuando $f(x) = c$ (integral de una constante).

2.3 Suma de potencias

Son la base para calcular integrales de polinomios.

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \tag{1}$$

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6} \tag{2}$$

$$\sum_{i=1}^n i^3 = \left(\frac{n(n+1)}{2} \right)^2 \tag{3}$$

2.4 Propiedad de desplazamiento o cambio de índice

$$\sum_{i=1}^n a_i = \sum_{i=0}^{n-1} a_{i+1}$$

Ayuda a reindexar sumas cuando se trabaja con particiones no uniformes.

2.5 Propiedad telescópica

$$\sum_{i=1}^n (a_i - a_{i-1}) = a_n - a_0$$

Fundamental para entender el Teorema Fundamental del Cálculo.

3 Conexión con la Integral Definida

Para calcular una integral definida mediante límites, seguimos estos pasos:

1. Dividimos $[a, b]$ en n subintervalos iguales: $\Delta x = \frac{b-a}{n}$.
2. Elegimos el punto muestra, por ejemplo el extremo derecho: $x_i = a + i\Delta x$.
3. Formamos la suma de Riemann:

$$S_n = \sum_{i=1}^n f(a + i\Delta x) \Delta x$$

4. Aplicamos las propiedades de sumatoria para expresar S_n en términos de n .
5. Tomamos el límite cuando $n \rightarrow \infty$.

3.1 Ejemplo clave: $\int_0^1 x^2 dx$

$$\begin{aligned} S_n &= \sum_{i=1}^n \left(\frac{i}{n}\right)^2 \cdot \frac{1}{n} \\ &= \frac{1}{n^3} \sum_{i=1}^n i^2 \\ &= \frac{1}{n^3} \cdot \frac{n(n+1)(2n+1)}{6} \\ &= \frac{(n+1)(2n+1)}{6n^2} \end{aligned}$$

Tomando límite:

$$\lim_{n \rightarrow \infty} S_n = \lim_{n \rightarrow \infty} \frac{(n+1)(2n+1)}{6n^2} = \frac{2}{6} = \frac{1}{3}$$

Por lo tanto,

$$\int_0^1 x^2 dx = \frac{1}{3}$$

4 Propiedades Adicionales Útiles

- **Suma de una progresión aritmética:** $\sum_{i=1}^n (a + (i-1)d) = \frac{n}{2}(2a + (n-1)d)$
- **Suma de exponenciales:** $\sum_{i=1}^n r^i = \frac{r^{n+1}-r}{r-1}$ (para $r \neq 1$)

5 Conclusión

Las propiedades de la sumatoria son el puente algebraico que conecta la suma finita con el límite infinito. Dominarlas permite:

- Calcular integrales definidas a partir de su definición.
- Comprender la convergencia de sumas de Riemann.
- Justificar la linealidad y aditividad de la integral.

La integral definida no es más que el límite de una suma bien estructurada; las propiedades presentadas aquí son las herramientas que hacen posible ese paso al infinito.

Uso del ciclo for en Python

Aplicado a Listas, Arreglos, Matrices y Tuplas

Documento Explicativo

1 de septiembre de 2026

1. Introducción

El ciclo `for` en Python es una estructura de control fundamental que permite iterar sobre secuencias de elementos. Su sintaxis es clara y legible, lo que lo convierte en una herramienta poderosa para recorrer colecciones de datos como listas, arreglos, matrices y tuplas.

En este documento exploraremos cómo aplicar el ciclo `for` en cada una de estas estructuras, acompañado de ejemplos prácticos que faciliten su comprensión.

2. Listas

2.1. Descripción

Una lista en Python es una colección ordenada y mutable de elementos. Puede contener datos de diferentes tipos y es una de las estructuras más utilizadas.

2.2. Ejemplo

Recorrer una lista de números e imprimir su cuadrado.

```
1 # Definimos una lista de numeros
2 numeros = [2, 4, 6, 8, 10]
3
4 # Iteramos sobre cada elemento de la lista
5 for num in numeros:
6     cuadrado = num ** 2
7     print(f"El cuadrado de {num} es {cuadrado}")
```

Listing 1: Uso de `for` con listas

3. Arreglos (Arrays)

3.1. Descripción

En Python, los arreglos se manejan comúnmente mediante el módulo `array` o, más frecuentemente, con la librería `NumPy`. Un arreglo es una estructura homogénea que permite operaciones vectorizadas de manera eficiente.

3.2. Ejemplo con array nativo

Recorrer un arreglo de números enteros y sumar sus elementos.

```
1 from array import array
2
3 # Creamos un arreglo de enteros
4 edades = array('i', [18, 22, 25, 30, 28])
5
6 suma_edades = 0
7 for edad in edades:
8     suma_edades += edad
9
10 print(f"La suma de las edades es: {suma_edades}")
```

Listing 2: Uso de `for` con arreglos nativos

3.3. Ejemplo con NumPy

Recorrer un arreglo NumPy y mostrar cada elemento multiplicado por 2.

```
1 import numpy as np
2
3 # Creamos un arreglo NumPy
4 precios = np.array([15.5, 20.0, 7.25, 12.75])
5
6 for precio in precios:
7     print(f"Precio con impuesto: {precio * 1.16:.2f}")
```

Listing 3: Uso de `for` con arreglos NumPy

4. Matrices

4.1. Descripción

Una matriz es un arreglo bidimensional (tabla de filas y columnas). En Python se puede representar como una lista de listas o mediante `NumPy`.

4.2. Ejemplo con lista de listas

Recorrer una matriz e imprimir sus elementos en formato tabla.

```

1 # Definimos una matriz (lista de listas)
2 matriz = [
3     [1, 2, 3],
4     [4, 5, 6],
5     [7, 8, 9]
6 ]
7
8 print("Matriz:")
9 for fila in matriz:
10     for elemento in fila:
11         print(elemento, end=" ")
12     print() # Salto de linea al terminar cada fila

```

Listing 4: Uso de for anidado con matrices

4.3. Ejemplo con NumPy

Recorrer una matriz NumPy y calcular la suma de todos sus elementos.

```

1 import numpy as np
2
3 # Creamos una matriz NumPy 2x3
4 matriz_np = np.array([[1, 2, 3], [4, 5, 6]])
5
6 suma_total = 0
7 for fila in matriz_np:
8     for columna in fila:
9         suma_total += columna
10
11 print(f"La suma de todos los elementos de la matriz es: {suma_total}")

```

Listing 5: Uso de for con matrices NumPy

5. Tuplas

5.1. Descripción

Una tupla es una colección ordenada e inmutable de elementos. Es similar a una lista, pero una vez creada no se puede modificar, lo que la hace ideal para datos que no deben cambiar.

5.2. Ejemplo

Recorrer una tupla de coordenadas y mostrar su información.

```

1 # Definimos una tupla de coordenadas (x, y)
2 puntos = ((1, 2), (3, 4), (5, 6), (7, 8))
3
4 print("Lista de puntos:")
5 for x, y in puntos:

```

```
6 print(f"Punto: ({x}, {y}) - Distancia al origen: { (x**2 + y**2)  
    **0.5:.2f}")
```

Listing 6: Uso de `for` con tuplas

6. Conclusión

El ciclo `for` es una herramienta versátil que se adapta perfectamente a las diferentes estructuras de datos en Python:

- **Listas:** ideales para colecciones mutables y heterogéneas.
- **Arreglos:** eficientes para datos homogéneos y operaciones matemáticas.
- **Matrices:** permiten trabajar con datos bidimensionales.
- **Tuplas:** útiles para datos inmutables y desempaqueado sencillo.

Cada uno de estos ejemplos muestra la flexibilidad del ciclo `for` para iterar y procesar información de manera clara y eficiente.

```

x=0
s=1
for a in range(1,21):
    # print(a)
    s += 5
    s=s+a
    print(f"El valor de a: {a} y el valor de s después de incrementarlo a veces es: {s}")

```

```

El valor de a: 1 y el valor de s después de incrementarlo a veces es: 7
El valor de a: 2 y el valor de s después de incrementarlo a veces es: 14
El valor de a: 3 y el valor de s después de incrementarlo a veces es: 22
El valor de a: 4 y el valor de s después de incrementarlo a veces es: 31
El valor de a: 5 y el valor de s después de incrementarlo a veces es: 41
El valor de a: 6 y el valor de s después de incrementarlo a veces es: 52
El valor de a: 7 y el valor de s después de incrementarlo a veces es: 64
El valor de a: 8 y el valor de s después de incrementarlo a veces es: 77
El valor de a: 9 y el valor de s después de incrementarlo a veces es: 91
El valor de a: 10 y el valor de s después de incrementarlo a veces es: 106
El valor de a: 11 y el valor de s después de incrementarlo a veces es: 122
El valor de a: 12 y el valor de s después de incrementarlo a veces es: 139
El valor de a: 13 y el valor de s después de incrementarlo a veces es: 157
El valor de a: 14 y el valor de s después de incrementarlo a veces es: 176
El valor de a: 15 y el valor de s después de incrementarlo a veces es: 196
El valor de a: 16 y el valor de s después de incrementarlo a veces es: 217
El valor de a: 17 y el valor de s después de incrementarlo a veces es: 239
El valor de a: 18 y el valor de s después de incrementarlo a veces es: 262
El valor de a: 19 y el valor de s después de incrementarlo a veces es: 286
El valor de a: 20 y el valor de s después de incrementarlo a veces es: 311

```

```

print(f"Ingresar un valor real y entero positivo")
x=int(input("Ingresar el valor entero positivo: "))

for a in range(10,31):
    y=x+2
    z=y+1
    x += 1
    y1=y+x
    # print(f"Los valores son: {[x,a,y,z,x,y1]}")
    print(f"Los valores son: [x={x},a={a},y={y},z={z},y1={y1}]")

print(f"El valor de y1 es {y1}")

```

```

Ingresar un valor real y entero positivo
Ingresar el valor entero positivo: 2
Los valores son: [x=3,a=10,y=4,z=5,y1=7]
Los valores son: [x=4,a=11,y=5,z=6,y1=9]
Los valores son: [x=5,a=12,y=6,z=7,y1=11]
Los valores son: [x=6,a=13,y=7,z=8,y1=13]
Los valores son: [x=7,a=14,y=8,z=9,y1=15]
Los valores son: [x=8,a=15,y=9,z=10,y1=17]
Los valores son: [x=9,a=16,y=10,z=11,y1=19]
Los valores son: [x=10,a=17,y=11,z=12,y1=21]
Los valores son: [x=11,a=18,y=12,z=13,y1=23]
Los valores son: [x=12,a=19,y=13,z=14,y1=25]
Los valores son: [x=13,a=20,y=14,z=15,y1=27]
Los valores son: [x=14,a=21,y=15,z=16,y1=29]
Los valores son: [x=15,a=22,y=16,z=17,y1=31]
Los valores son: [x=16,a=23,y=17,z=18,y1=33]
Los valores son: [x=17,a=24,y=18,z=19,y1=35]
Los valores son: [x=18,a=25,y=19,z=20,y1=37]
Los valores son: [x=19,a=26,y=20,z=21,y1=39]

```



```
Los valores son: [x=20,a=27,y=21,z=22,y1=41]
Los valores son: [x=21,a=28,y=22,z=23,y1=43]
Los valores son: [x=22,a=29,y=23,z=24,y1=45]
Los valores son: [x=23,a=30,y=24,z=25,y1=47]
El valor de y1 es 47
```

```
n=0
for i in range(1,16):
    x=2*n+1
    n += 1
    print(f"El valor de n es: {n} y el valor de x es: {x}")
```

```
El valor de n es: 1 y el valor de x es: 1
El valor de n es: 2 y el valor de x es: 3
El valor de n es: 3 y el valor de x es: 5
El valor de n es: 4 y el valor de x es: 7
El valor de n es: 5 y el valor de x es: 9
El valor de n es: 6 y el valor de x es: 11
El valor de n es: 7 y el valor de x es: 13
El valor de n es: 8 y el valor de x es: 15
El valor de n es: 9 y el valor de x es: 17
El valor de n es: 10 y el valor de x es: 19
El valor de n es: 11 y el valor de x es: 21
El valor de n es: 12 y el valor de x es: 23
El valor de n es: 13 y el valor de x es: 25
El valor de n es: 14 y el valor de x es: 27
El valor de n es: 15 y el valor de x es: 29
```

```
import numpy as np

arreglo = np . array ([10 , 20 , 30 , 40 , 50])
suma = 0
for valor in arreglo :
    suma += valor

print (f"La suma del arreglo es :{suma}")
```

La suma del arreglo es :150

```
A=range(1,21)
arreglo=A
suma = 0
for valor in arreglo:
    suma += valor

print (f"La suma del arreglo es :{suma}")
```

La suma del arreglo es :210

```
matriz = [
    [1 , 2 , 3] ,
    [4 , 5 , 6] ,
    [7 , 8 , 9]
]

for fila in matriz :
    for elemento in fila :
        print ( elemento , end = " \n" )

print () # Salto de línea al terminar cada fila
```

1
2
3
4
5
6
7
8
9